

Uczenie się ze wzmocnieniem

WSI – ćwiczenie 6

Autor: Kacper Skrodzki

Wstęp do eksperymentów

W ramach szóstego ćwiczenia należało zaimplementować algorytm **Q-learning** z epizodami oraz ϵ -zachłanną strategią wyboru akcji, który ma zostać użyty do wytrenowania agenta rozwiązującego problem *Four Rooms*, z pakietu *minigrid*.

Eksperymenty dotyczące problemu *Four Rooms* będą miały na celu zbadanie wpływu różnych parametrów agenta:

- **Współczynnik dyskontowania** – γ (gamma)
- **Szybkość uczenia** – β (beta; learning rate)
- **Parametr eksploracji** – ϵ (epsilon)

na jego czas znajdowania optymalnej strategii oraz jakość znalezionej strategii.

Dodatkowo, w ramach poleceń dla chętnych wprowadziłem opcję trenowania agenta z losowych pozycji początkowych agenta i pozycji celu, oraz zaimplementowałem algorytm **SARSA**. Też go omówię w kontekście parametrów i porównam z Q-learning.

Eksperymenty – Q-learning

W eksperymentach dla Q-learning-u wybrałem trzy zestawy parametrów:

- **Zestaw standardowy** – $\beta = 0.1$ | $\gamma = 0.99$ | $\epsilon = 0.1$
- **Zestaw ‘ostrożny’** – $\beta = 0.05$ | $\gamma = 0.99$ | $\epsilon = 0.05$
- **Zestaw ‘agresywny’** – $\beta = 0.5$ | $\gamma = 0.9$ | $\epsilon = 0.3$

Każdy z zestawów ma reflektować różne podejście do dylematu **eksploracja**, a **eksploatacja**.

Eksploracja ma służyć do mapowania całej dostępnej planszy i znajdowania nowych możliwych tras.

Eksploatacja ma służyć do wybierania najlepszych ruchów, aby móc udoskonalać trasę do końcowego pola.

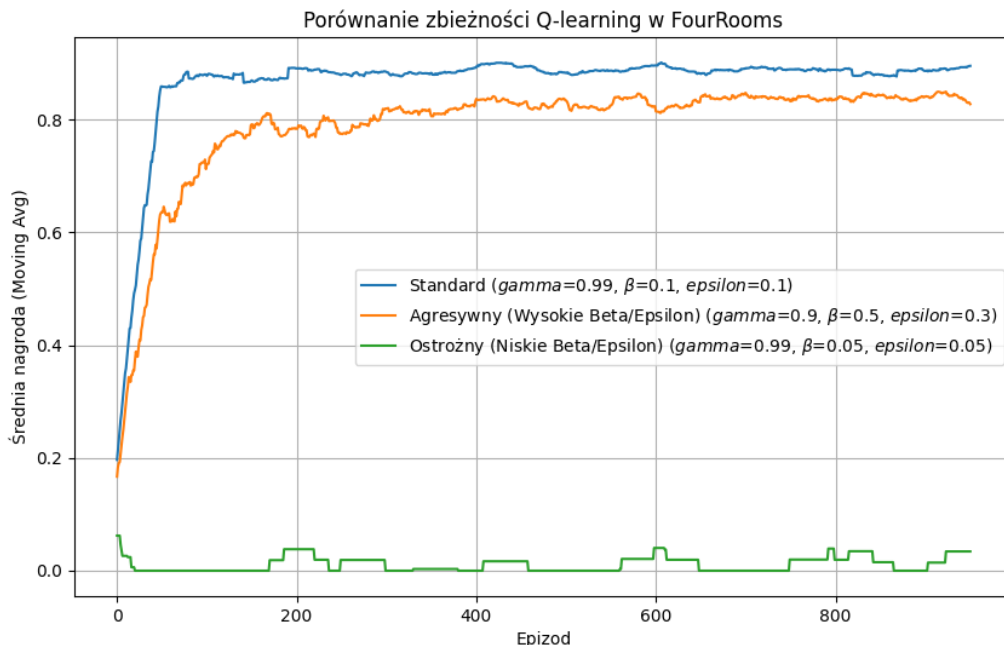
Zestaw standardowy ma w sobie zbalansowane podejście do eksploracji i eksploatacji, zestaw 'ostrożny' skupia się silnie na eksploatacji, a zestaw 'agresywny' faworyzuje eksplorację.

Aby ukazać pełne spektrum efektywności poszczególnych zestawów, poniższe wykresy będą ukazywać skuteczność agentów w trakcie trwania treningu dla różnych random seedów.

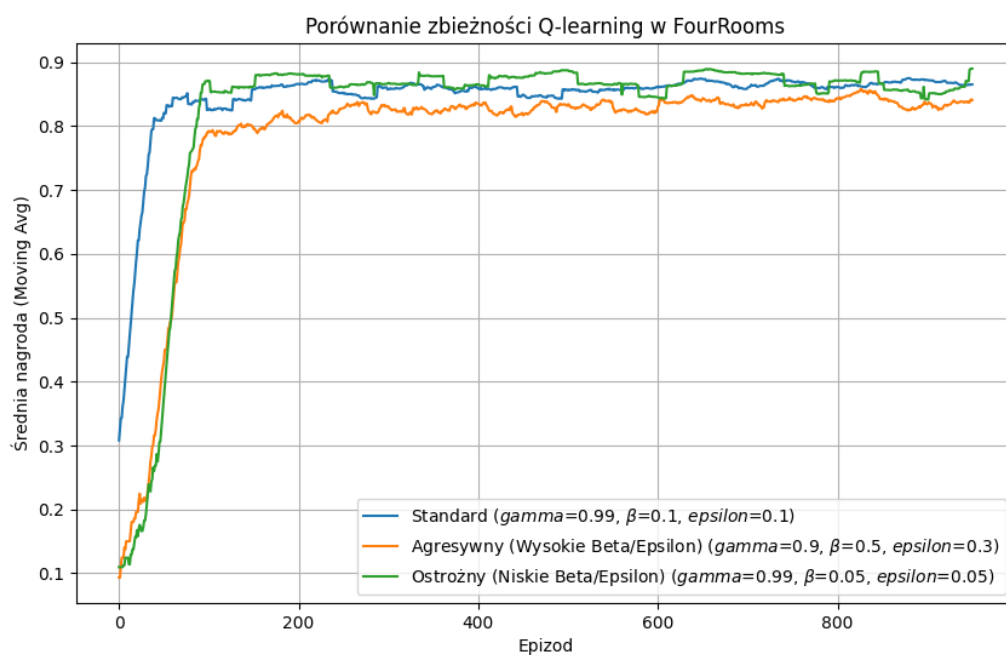
Reszta parametrów treningowych:

- Maksymalna liczba kroków w epizodzie – 100 kroków
- Liczba epizodów – 1000 epizodów
- Random seed-y – [35, 37, 42, 53, 77]
- Pozycja początkowa agenta – (6, 6)
- Pozycja mety – (2, 2)

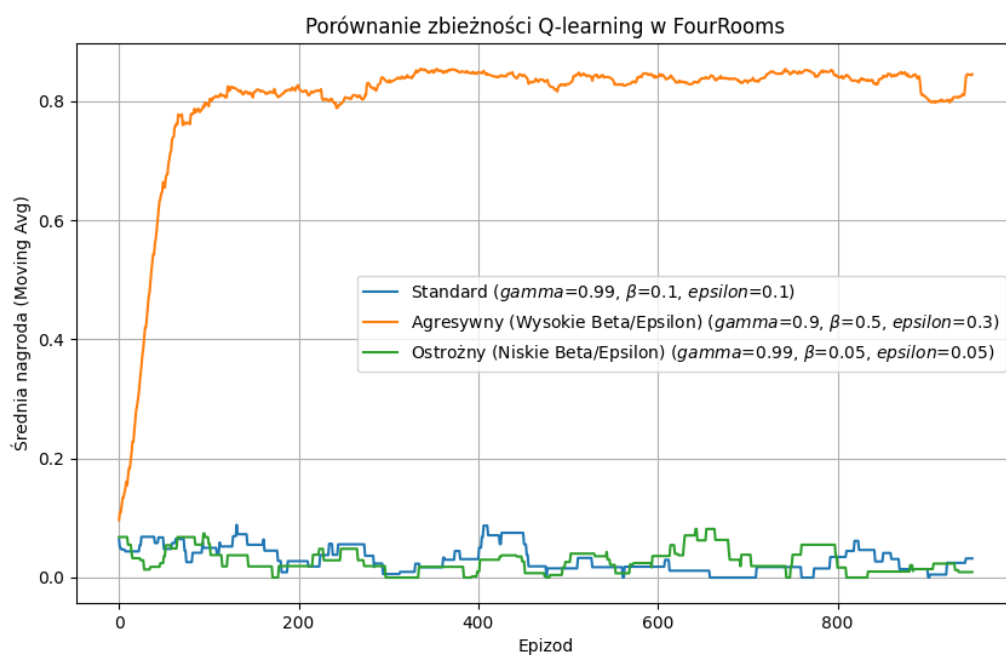
Adnotacja do random seed-ów - po testach okazało się, że nawet z random seed-ami nie da się odtworzyć tego samego wyniku trenowania. Najprawdopodobniej wynika to z faktu, że ilość operacji losowych oraz natura poruszania się po planszy powoduje, że nie da się odtworzyć dokładnego treningu.



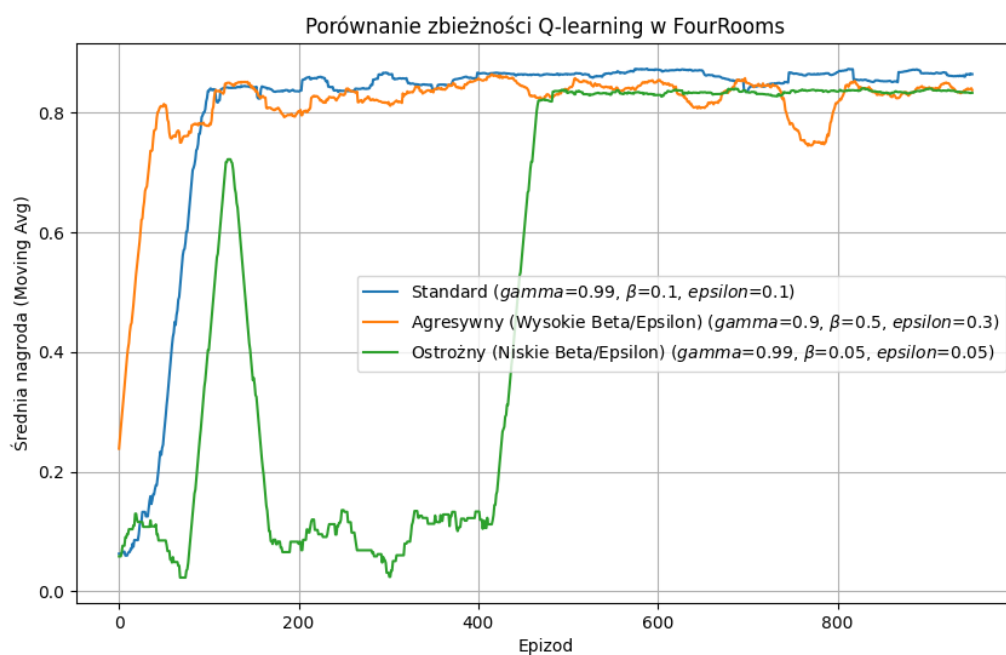
Random seed = 35



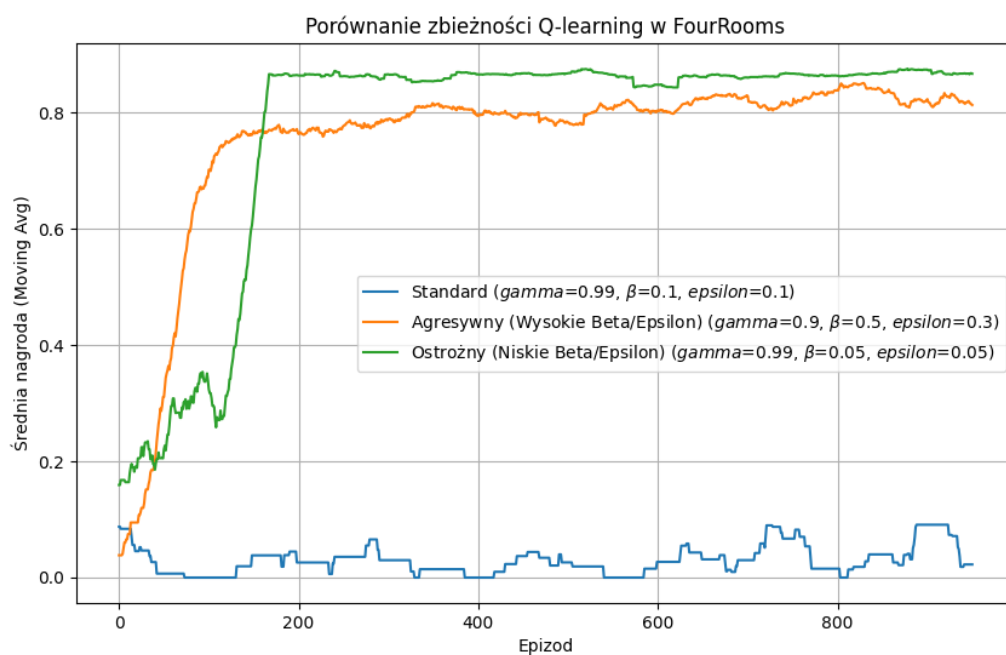
Random seed = 37



Random seed = 42



Random seed = 53



Random seed = 77

Wnioski:

Jak widać z powyższych wykresów każdy z zestawów zachowuje się różnie od pozostałych. Niestety wiele wyników zależy od losowości tego, czy agent trafi na pole z nagrodą, czy nie. Jednakże nadal można zauważyć pewne powtarzające się trendy.

Przykładowo, zestaw ‘agresywny’ prawie zawsze znajdzie odpowiednią trasę do celu i to w bardzo szybkim tempie. Jednakże nie będzie w stanie osiągnąć najlepszej możliwej, gdyż jego nacisk na eksplorację powoduje, że za często ‘obija’ się o niepotrzebnie i marnuje tym możliwość ulepszenia trasy. Widać to na wykresie – często oscyluje już po osiągnięciu dobrej strategii.

Zestaw ‘ostrożny’ natomiast jest zgoła inny. Statystycznie potrzebuje więcej czasu na to, aby mógł odnaleźć cel albo nawet nie jest w stanie znaleźć celu w czasie treningu. Wynika to z faktu, że preferuje on skupienie się na eksploatacji niż na eksploracji. Takie podejście początkowo powoduje zastój, gdyż agent nie jest w stanie znaleźć pola z nagrodą. Jednakże w momencie kiedy uda mu się znaleźć pole z nagrodą, szybko wyszukuje najlepszą trasę o lepszej jakości niż inne zestawy są w stanie osiągnąć. Choć oczywiście zdarzają się wyjątki.

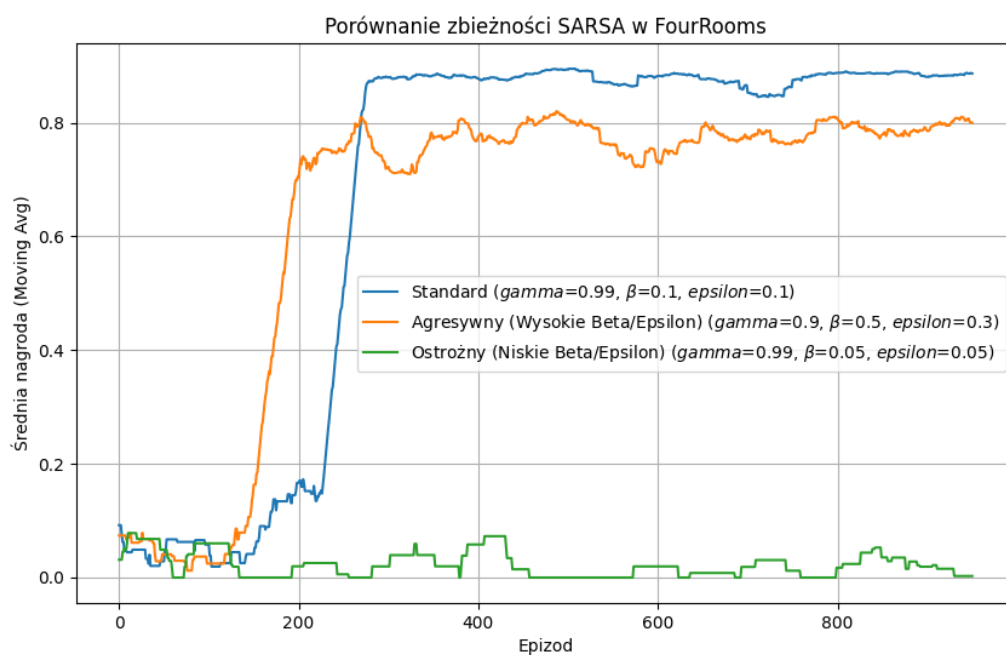
Zestaw standardowy łączy w sobie cechy zestawów ‘agresywny’ i ‘ostrożny’. W większości potrafi znaleźć cel dość w szybkim czasie, jest stabilny w optymalizowaniu drogi do celu i osiąga przy tym przyzwoitą jakość trasy. Jednakże czasami nie potrafi znaleźć jakiegokolwiek pola z nagrodą, lecz jest to już bolączka losowości algorytmu oraz natury problemu *Four Rooms* (wąskie gardła, które łatwo ominąć).

Losowy start – omówienie

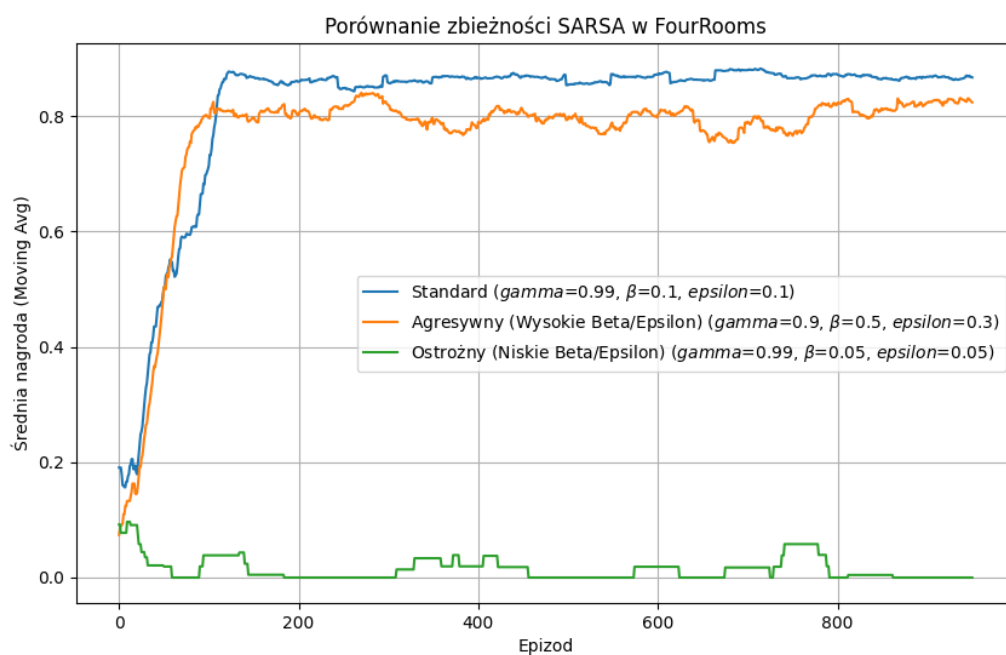
W implementacji jest możliwość trenowania agenta dla losowych pól startu i celu co epizod, lecz jest to dość czasochłonny proces. Wynika to z faktu, że w stanach zapamiętywanych przez agenta pojawiają się pola dotyczące pozycji celu, aby móc mapować optymalne ruchy dla konkretnego przypadku planszy. Niestety takie rozwiązanie powoduje, że czas w którym trzeba byłoby trenować agenta wzrósł z 1000 epizodów, do wartości rzędem kilku milionów epizodów.

Eksperymenty – SARSA

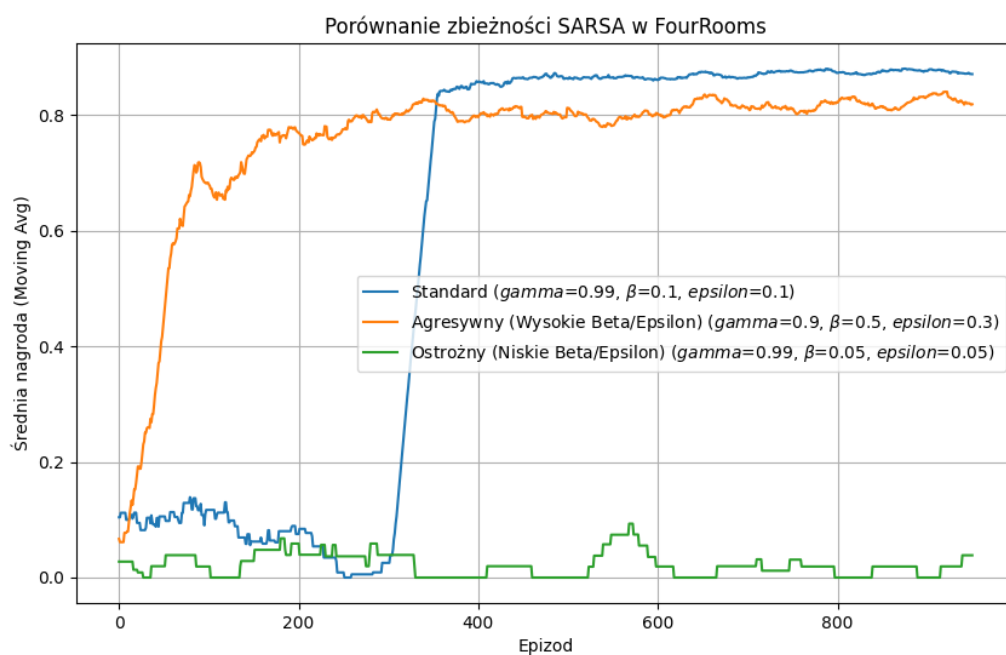
W eksperymentach dla SARSA użyłem te same zestawy parametrów dla agenta i dla trenowania. Dodatkowo przedstawiłem bezpośrednie zestawienie między **Q-learning**, a **SARSA**.



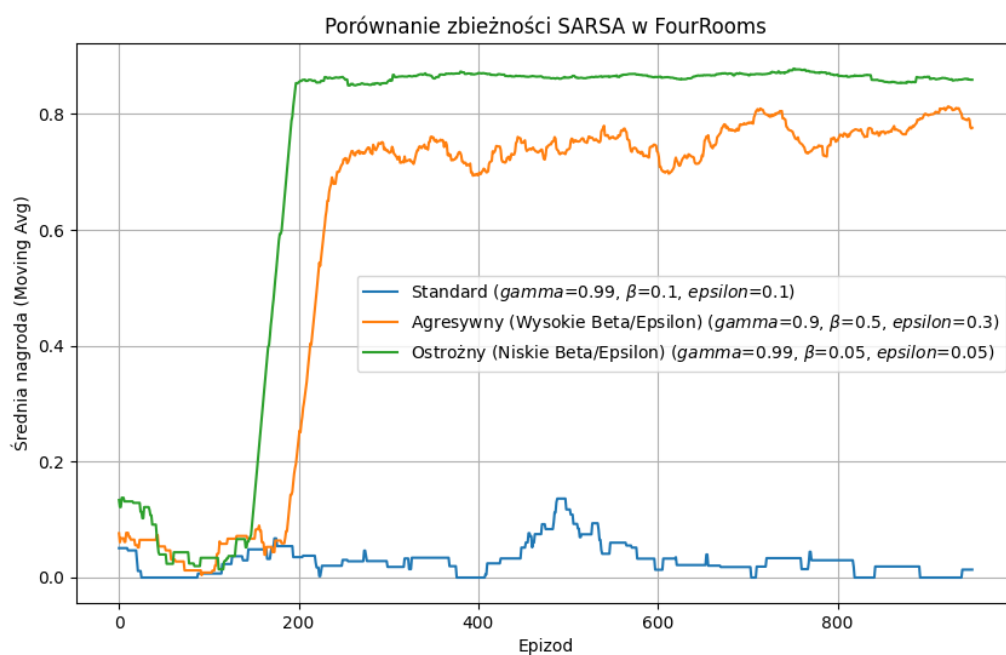
Random seed = 35



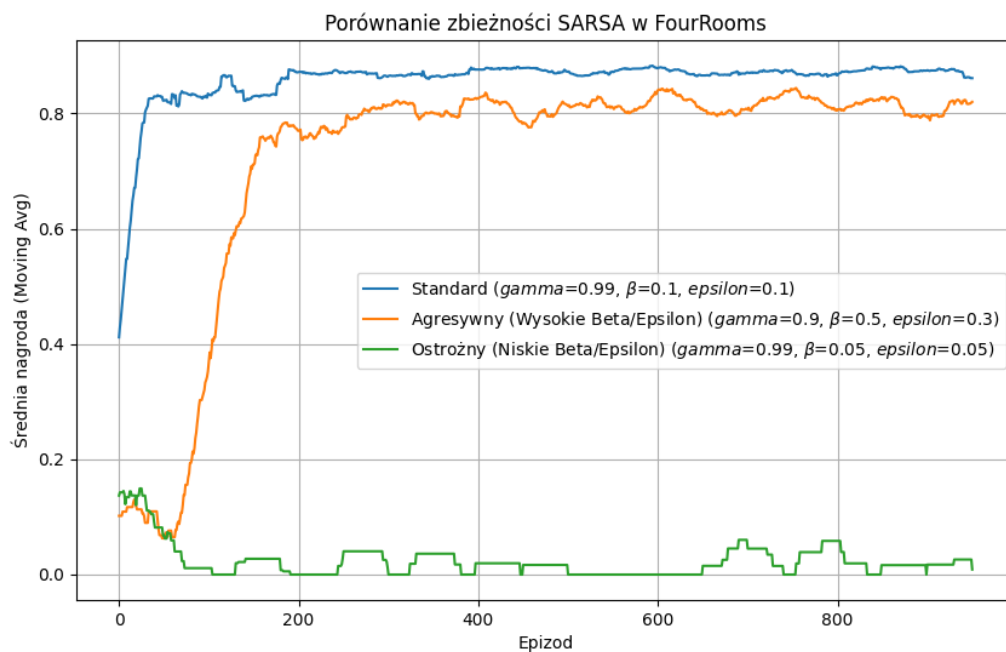
Random seed = 37



Random seed = 42



Random seed = 53

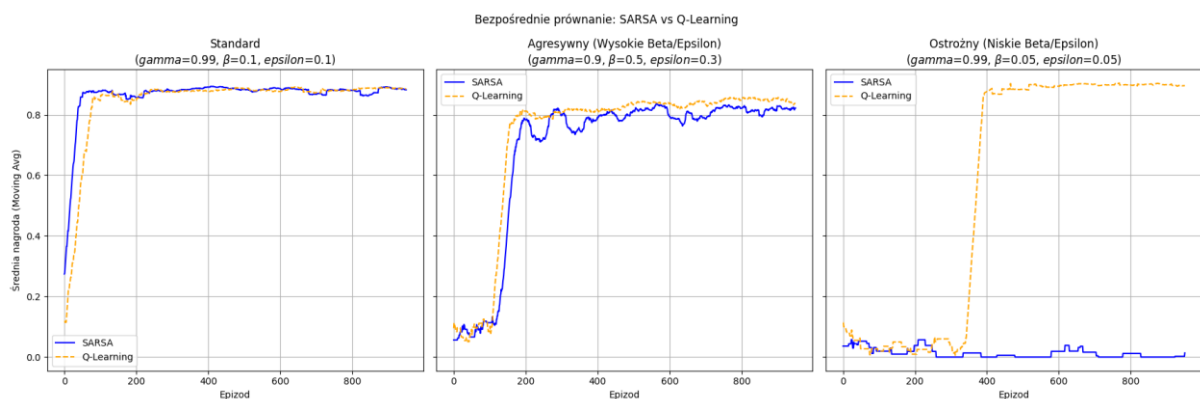


Random seed = 77

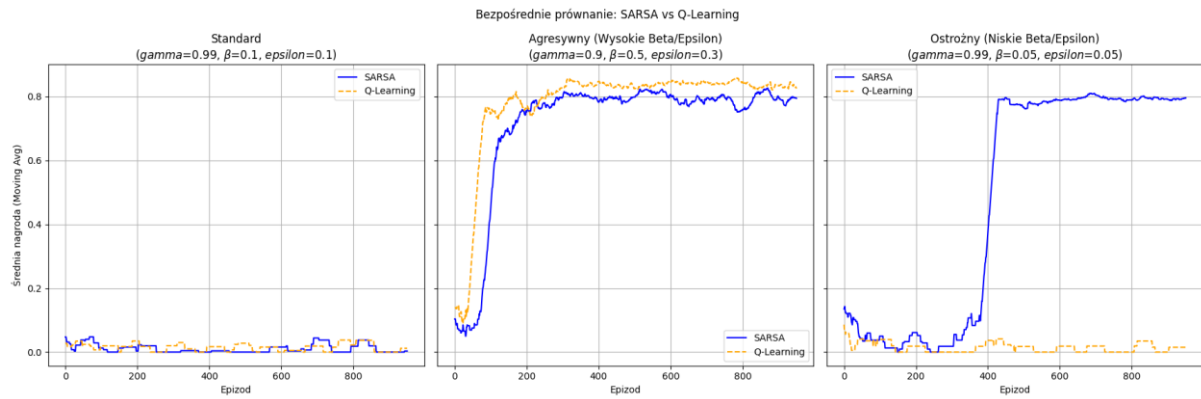
Wnioski:

W przypadku SARSA nie ma drastycznych różnic pomiędzy zachowaniem poszczególnych zestawów względem tego co już wcześniej omawiałem. Warto jednak zaznaczyć, że natura SARSA powoduje, że agent ma tendencję do bycia ogólnie ‘ostrożnym’. Dłużej mu zajmuje znalezienie celu, lecz ma mniejszą tendencję do zapominania tego co już się nauczył i mniej oscyluje.

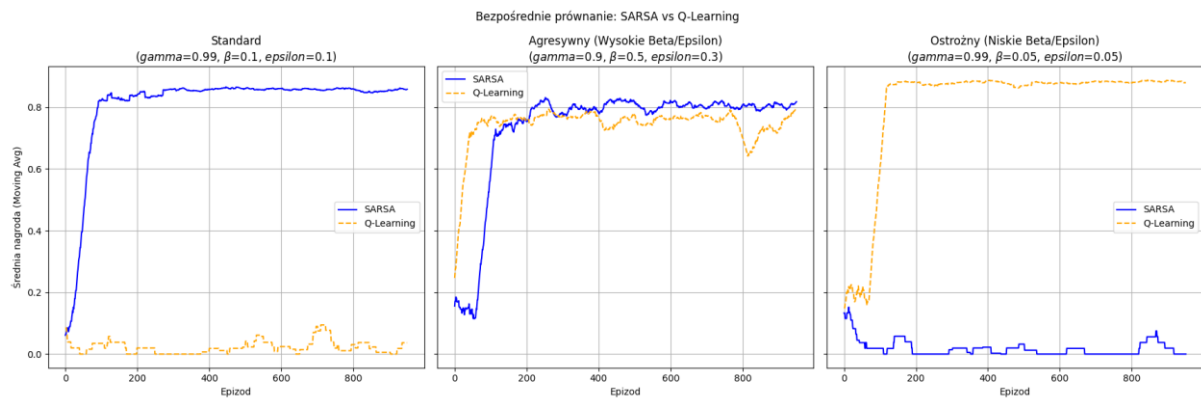
Q-learning, a SARSA



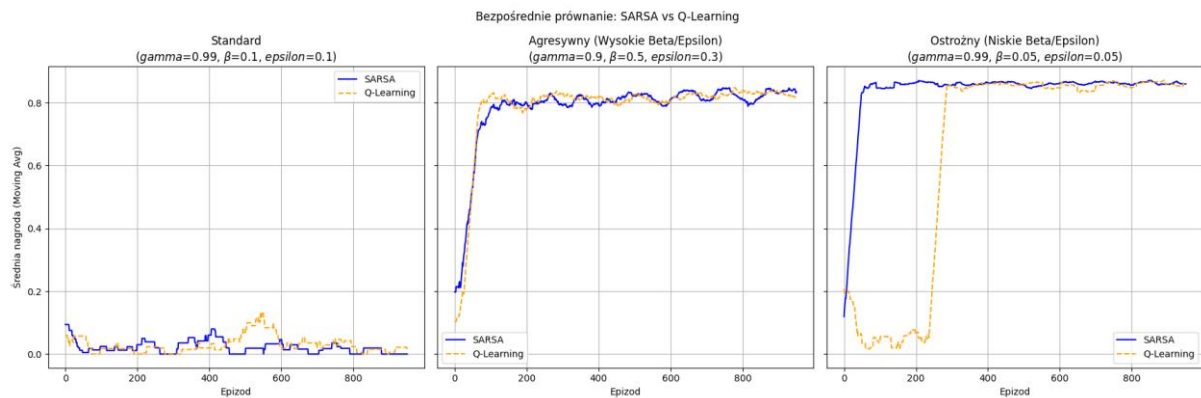
Random seed = 35



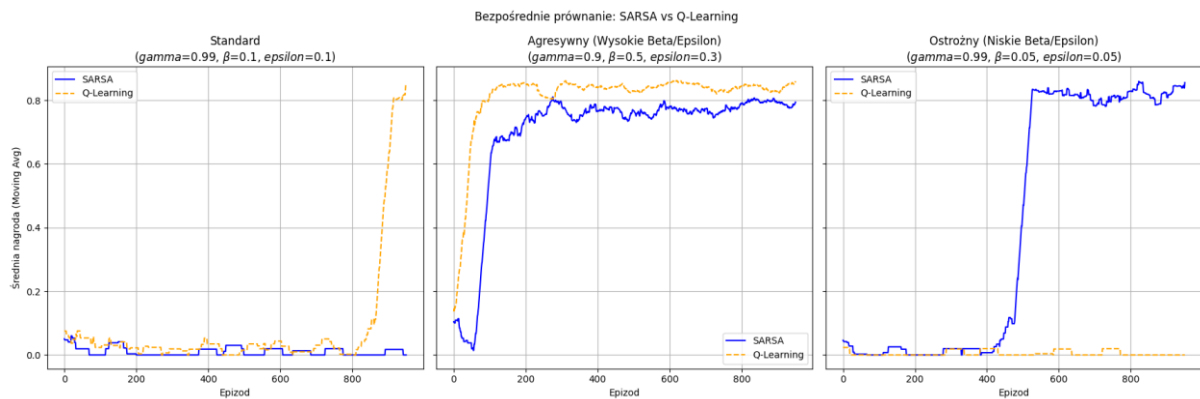
Random seed = 37



Random seed = 42



Random seed = 53



Random seed = 77

Wnioski:

Z porównania Q-learning z SARSA wynika jasno, że ten pierwszy statystycznie szybciej znajduje cel i dobrą strategię, a ten drugi wolniej, lecz stabilniej. Jednakże każdy z nich jest podatny na wąskie gardła problemu *Four Rooms*.

Podsumowanie

Prosta implementacja Q-learning i SARSA jest bardzo podatna na dobrane parametry. Odpowiedni bilans między eksploracją, a eksploatacją jest kluczem do osiągnięcia dobrych wyników. Jednakże dla problemu *Four Rooms*, samo dobranie parametrów nie jest w stanie przeciwdziałać problemowi wąskich gardeł w planszy. Implementacja *Epsilon Decay* mogłaby zmniejszyć wpływ tego problemu.